Lesson 8: Under Pressure

Unity

VIDEO GAME CREATION

This week: give the enemies you dodge last week actual stakes. An enemy reaching the bottom now costs a life - lose all 3, and "Game Over" appears.

Open your own [YourName]Game project from Lesson 7 - don't start a new one.

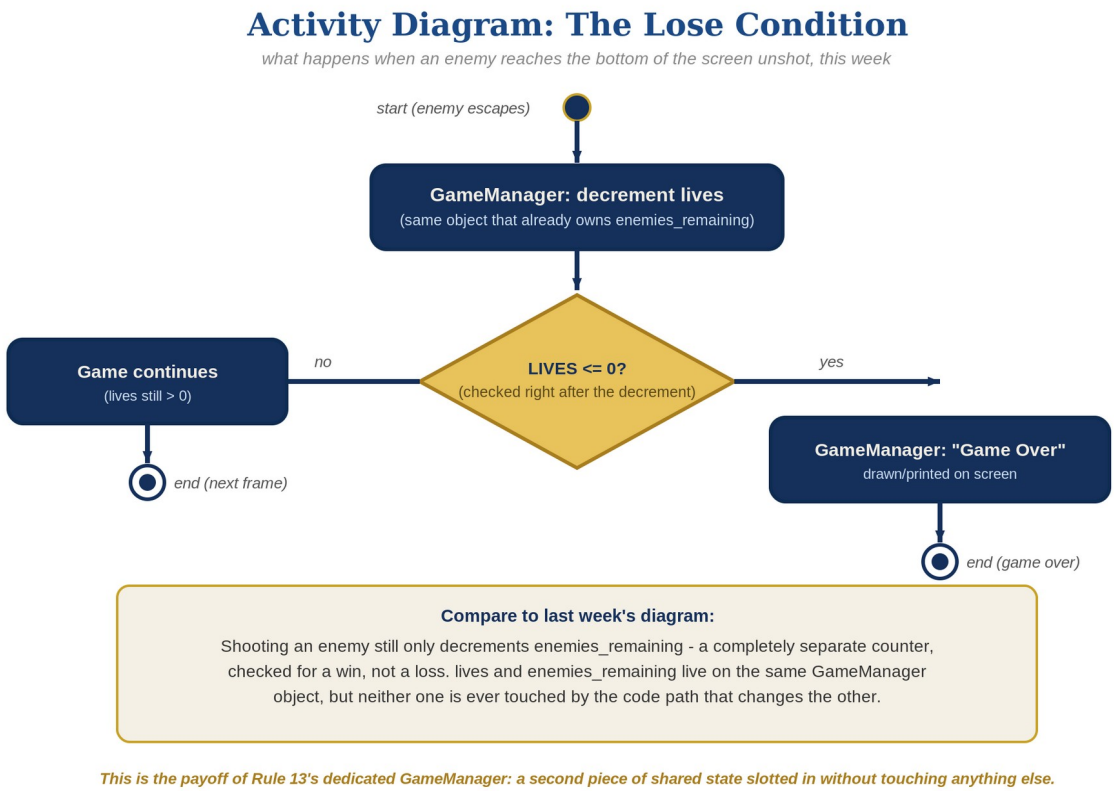
Tip: the core exercise needs no new assets - this reuses last week's escape-cleanup code and just gives it a real consequence. For Level 3 sound, use the provided sfx_lifelost.wav and sfx_gameover.wav from the Lesson 8 Assets folder.

Setup

1. Open your [YourName]Game project from Lesson 7.

Level 1 - Minimal (MVP)

Every enemy that reaches the bottom unshot now costs the player one life, tracked starting at 3. The moment lives reach zero, "Game Over" appears on screen.



This is the payoff of Rule 13's dedicated GameManager: a second piece of shared state slotted in without touching anything else.

The lose-condition logic behind this week's exercise - the same shape in every engine, whatever the exact code or blocks look like.

2. In GameManager.cs, add a public static int lives = 3, a public static bool hasLost = false, and a static method RegisterEscape() that decrements lives and sets hasLost at zero (code below).
3. In Enemy.cs's Update(), find the escape branch you built last week. Right before Destroy(gameObject), call GameManager.RegisterEscape().
4. In GameManager.cs's OnGUI(), add a check that draws "Game Over" when hasLost is true, alongside the existing hasWon check.
5. Press Play and let all 3 lives run out by letting enemies escape unshot - "Game Over" should appear the instant the third one escapes.

Highlighted lines are new this week; everything else is code you already have.

```
// GameManager.cs - add fields and a new method:
public static int lives = 3;
public static bool hasLost = false;

public static void RegisterEscape() {
    lives--;
    if (lives <= 0) hasLost = true;
}

// GameManager.cs's OnGUI() - add alongside the hasWon check:
if (hasLost) {
    GUI.Label(new Rect(Screen.width/2 - 50, 90, 200, 30), "Game Over");
}

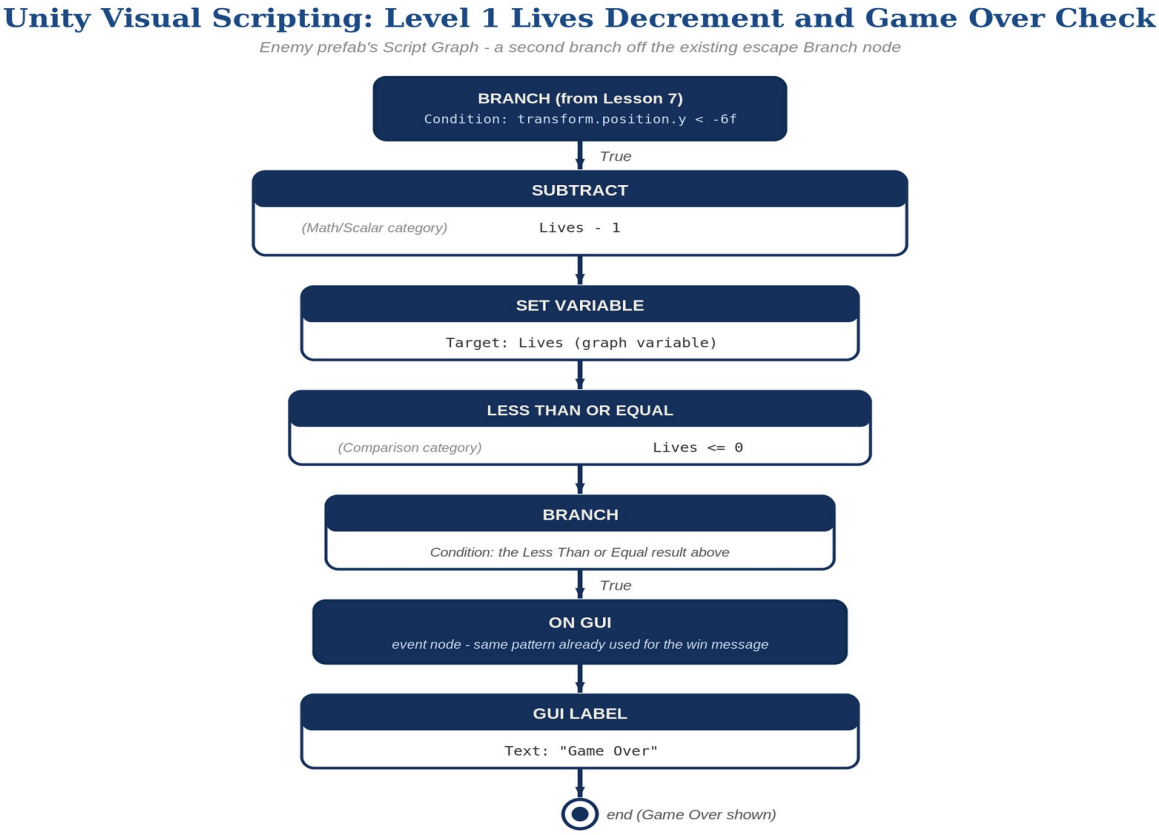
// Enemy.cs's Update() - escape branch (revised):
```

```
if (transform.position.y < -6f) {
    GameManager.RegisterEscape();
    Destroy(gameObject);
}
```

Alternate Path: Visual Scripting

The same lives-tracking pattern works in a Script Graph instead of C#.

6. On the Enemy prefab's graph, find the escape Branch from Lesson 7, right before its Destroy node. Add a "Subtract" node (Math/Scalar category) taking the Lives graph variable and 1, feeding into a "Set Variable" node targeting Lives - the node-graph equivalent of lives--.
7. Feed Lives into a "Less Than or Equal" node (Comparison category) against 0, feeding a Branch node.
8. On that Branch's True output, add an "On GUI" event node with a GUI Label node showing "Game Over", the same pattern already used for the win message.
9. Confirm this new logic only touches the Lives graph variable, never the separate variable used for enemies-remaining/win-tracking.



The Enemy prefab's Script Graph - the node-for-node equivalent of the Level 1 lives-tracking logic.

Level 2 - Complete (Alpha/Beta)

Shooting an enemy never costs a life, and letting one escape never counts as a kill - the two counters stay completely independent.

- Shoot an enemy and confirm lives does NOT change - only RegisterKill(), called from Bullet.cs, should run.
- Let an enemy escape and confirm enemiesRemaining does NOT change - only RegisterEscape(), called from Enemy.cs's own Update(), should run.
- Confirm "Game Over" appears exactly once, exactly when lives reaches zero.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Visual: add a GUI.Label in OnGUI() showing the current lives value, updated every frame - a nice touch, though the full scoreboard-style UI is coming properly in Lesson 9.
- Sound: add two AudioClip fields to GameManager, assign the provided sfx_lifelost.wav and sfx_gameover.wav to them in the Inspector, and call AudioSource.PlayClipAtPoint() inside RegisterEscape() - the life-lost clip when a life is lost, the game-over clip instead the moment hasLost first becomes true.
- Structure: notice RegisterKill() and RegisterEscape() are two separate static methods, each called from exactly one script - this mirrors the exact separation that made GameManager safe from the Lesson 7 bug in the first place.

If Something's Wrong

- Lives decreases when you shoot an enemy: confirm RegisterEscape() is called only from Enemy.cs's Update(), never from Bullet.cs.
- "Game Over" never appears: confirm lives actually starts at 3 and RegisterEscape() is actually being called - add a temporary Debug.Log(lives) to check.
- Compile errors: confirm hasLost is declared as public static bool, matching hasWon's declaration style exactly.

Want More?

Student Resource List > Unity section, or learn.unity.com